

Firecracker

Lightweight virtualization for serverless applications

Agache et al.
USENIX NSDI 2020

Emanuele Volanti
ID 1761743

Mobile Applications & Cloud Computing
A.A. 2025 / 2026

The serverless model

- ◆ Customers upload **functions**, not servers
- ◆ AWS runs them **on demand** — billed per millisecond of execution
- ◆ No idle time, no server management
- ◆ The canonical example: **AWS Lambda**

THE CHALLENGE

Thousands of tenants share the same physical hosts.

The core tension

Two requirements that fight each other.

REQUIREMENT A

Strong isolation

Tenant A must never see — or attack — Tenant B.

vs

REQUIREMENT B

Low overhead

Millisecond startup, high density — many functions per host.

No existing technology delivered both. **Something new was needed.**

Why existing options fall short

TECHNOLOGY	ISOLATION	OVERHEAD
Linux containers Docker	 Shared kernel — one bug breaks everyone	 Fast and lightweight
Language VMs V8 isolates	 Strong, language-enforced	 Locked to a single language
Traditional VMs QEMU	 Strong, hardware-enforced	 Seconds to boot, hundreds of MB

None of them works for Lambda.

Firecracker: a minimal VMM

- ◆ Written from scratch in **Rust** for memory safety
- ◆ The **microVM** — a VM stripped to the essential
- ◆ Runs on top of **KVM**, Linux's hypervisor

ONLY 4 VIRTIO DEVICES

net

block

vsock

serial console

REMOVED ENTIRELY

BIOS

PCI

USB

graphics

ACPI

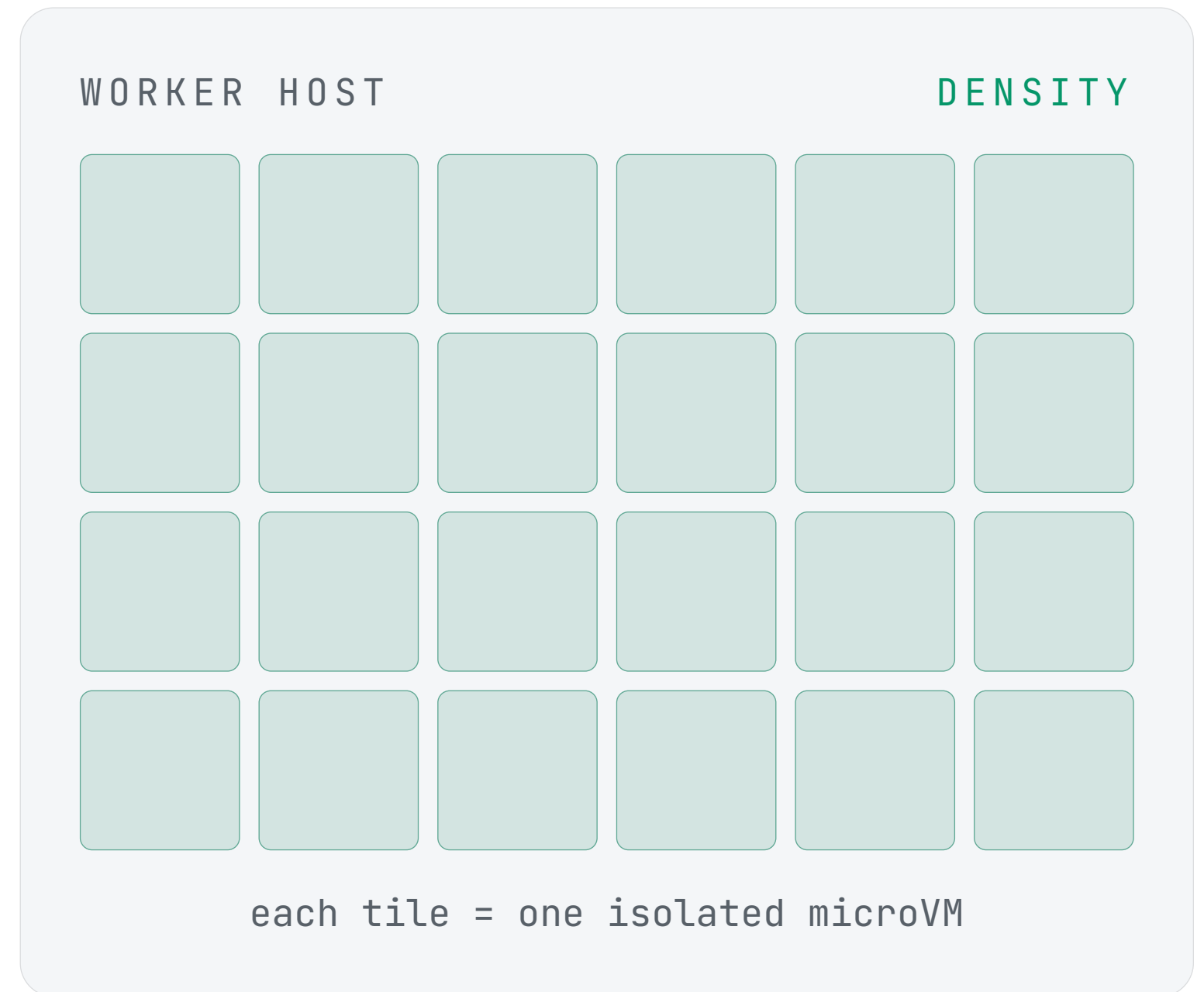
VM-grade isolation, **container-grade footprint.**

Integration with AWS Lambda

- ◆ 1 microVM per function instance — strict per-tenant boundary
- ◆ A **worker host** runs many microVMs — high density
- ◆ The **Jailer** sandboxes each process — namespaces, seccomp, cgroups, chroot

cold start → boot new

warm start → reuse



The numbers

<125_{ms}

Boot time, host to init

~5_{MiB}

Memory overhead per
microVM

~150_{/s}

MicroVMs started per
second, per host

2018

In production at AWS since

Trillions of Lambda invocations powered every month.

Takeaways

ACHIEVED	Both at once — VM-grade security and container-grade efficiency.
TRADEOFF	Flexibility was sacrificed — no GPU, no nested virtualization, no custom devices.
OPEN SOURCE	Adopted beyond AWS — Fly.io, Kata Containers, Weaveworks Ignite.
RELEVANCE	Serverless infrastructure must rethink classical OS isolation to scale economically.

Thank you

Questions?